

Conception et Implémentation d'une Base de Données Distribuée

pour une Plateforme e-Commerce B2B

Oracle XE 21c · Docker · PL/SQL · Triggers multi-maîtres

Hajar Elhilali ■ Yasmine Koulam

Cycle Ingénieur – Génie Informatique – 2^{ème} Année

Université Hassan 1^{er} – Faculté des Sciences et Techniques de Settat

Année universitaire 2025 – 2026

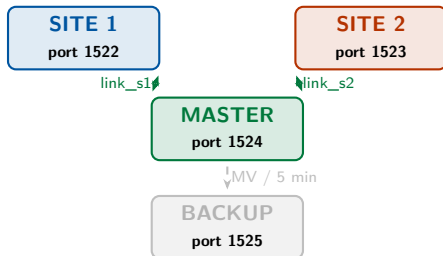
Problématique

Comment distribuer la table `LigneCommandes` sur plusieurs serveurs tout en garantissant la **cohérence** et la **transparence** pour les applications ?

Solution

- **Fragmentation horizontale** de `LigneCommandes`
- **2 scénarios** de distribution
- Synchronisation par **triggers PL/SQL**
- Identifiants uniques : **séquences modulo 3**
- Sauvegarde : **vues matérialisées**

Objectif : distribuer EShop B2B sur **4 nœuds Oracle XE 21c** conteneurisés (Docker), avec routage automatique des données.



Nœud	Port	Rôle
Master	1524	Base globale + seed
Site 1	1522	Fragment R_1
Site 2	1523	Fragment R_2
Backup	1525	Vues matérialisées

- **DB Links privés** + TNS partagé
- Compte `eshop_link` à moindre privilège
- Sites démarrent après le Master

Deux Scénarios de Fragmentation Horizontale

Scénario 1 – Dérivée

- $R_1 \rightarrow$ **Site 1** : cat=50 et qté>100
- $R_2 \rightarrow$ **Site 2** : cat=35 et qté>50

2 critères (catégorie + quantité)

Partition **partielle** (cat 10 reste sur Master)

Scénario 2 – Primaire

- $R_1 \rightarrow$ **Site 1** : qté $\geq$ 100
- $R_2 \rightarrow$ **Site 2** : qté < 100

1 seul critère (quantité)

Partition **totale** : $R_1 \cap R_2 = \emptyset$

Architecture **multi-maître** : on peut écrire sur **n'importe quel nœud**. Des **triggers PL/SQL** sur chaque nœud routent automatiquement chaque opération vers le bon fragment.

Réplication Multi-Maître – Écriture sur N'importe Quel Nœud

Trigger compound sur Site 1 (après INSERT)

```
-- 1. Propagation SYSTEMATIQUE au Master
INSERTligne@link_central(...);

-- 2. Routage croise vers Site 2 si la
--     ligne appartient au fragment R2
IF v_categ = 35 AND :NEW.quantite > 50 THEN
    INSERTligne@link_site2(...);
END IF;

-- 3. Si la ligne ne respecte PAS la regle
--     locale de Site 1 -> auto-suppression
IF NOT (v_categ = 50 AND qte > 100) THEN
    v_ids_to_delete.EXTEND; -- nettoyage
END IF;
```

Chaque nœud est un maître

- Écriture acceptée sur Master, **Site 1** ou **Site 2**
- Chaque nœud propage au **Master** (base globale)
- Puis redistribue vers l'**autre site** si besoin
- Garde **seulement** ses données, supprime le reste

Clés étrangères

Trigger **BEFORE INSERT** : rapatrie produit, commande et client depuis le Master avant la vérification des FK.

Migration Automatique entre Sites (UPDATE)

Trigger UPDATE sur Site 1 – Scénario 2

```
-- 1. Propager la MAJ au Master
updateligne@link_central(...);

-- 2. Si la quantite passe sous 100,
--    la ligne quitte le fragment R1
IF :NEW.quantite < 100 THEN
    -- migre vers Site 2 ...
    INSERTligne@link_site2(...);
    -- ... puis suppression locale
    v_ids_to_delete.EXTEND;
END IF;
```

4 cas selon le seuil 100

$\geq 100 \rightarrow \geq 100$	reste Site 1
$\geq 100 \rightarrow < 100$	Site 1 $\rightarrow$ Site 2
$< 100 \rightarrow \geq 100$	Site 2 $\rightarrow$ Site 1
$< 100 \rightarrow < 100$	reste Site 2

Cohérence cross-site

Une MAJ sur **Site 2** est propagée au Master, qui la redistribue vers **Site 1** : les 3 nœuds restent cohérents.

Anti-boucle : pkg_synchro

```
-- Flag de session sur chaque noeud :  
PACKAGE pkg_synchro AS  
    is_replicating BOOLEAN := FALSE;  
END;  
  
-- Les procedures INSERTligne /  
    updateligne /  
-- deleteligne posent le flag avant d'  
    écrire :  
pkg_synchro.is_replicating := TRUE;  
INSERT INTO lignecommandes (...);  
pkg_synchro.is_replicating := FALSE;  
-- => le trigger du noeud cible ne re-  
    propage  
-- pas (sinon boucle Master <-> Site  
    infinie)
```

Compound Trigger

- **AFTER EACH ROW** : collecte les IDs
- **AFTER STATEMENT** : DELETE en lot
- Évite l'erreur **ORA-04091** (table en mutation)

Journalisation

PRAGMA AUTONOMOUS_TRANSACTION : les erreurs sont enregistrées dans `sync_errors` indépendamment du ROLLBACK.

Problème

Génération autonome des IDs sur chaque nœud $\Rightarrow$ risque de **collisions** de clés primaires.



Solution : 3 voies disjointes

Nœud	$\text{mod}(\text{ID}, 3)$	Valeurs
Master	1	1, 4, 7...
Site 1	2	2, 5, 8...
Site 2	0	3, 6, 9...

Garantie

Les 3 ensembles sont **strictement disjoints** : aucune collision possible, à vie.

Démonstration

Q5 – Optimisation par index couvrant

Q6 – Requête distribuée (UNION ALL inter-sites)

Réalisations

- **2 scénarios** de fragmentation opérationnels
- Réplication par **triggers** + **pkg_synchro**
- **Migration automatique** (seuil de routage)
- Séquences **modulo 3** : zéro collision
- Sauvegarde par **vues matérialisées**
- Optimisation Q5 (−28%) et Q6 distribuée

Perspectives

- Fragmentation verticale
- Réplication asynchrone
- Load balancing inter-sites

Merci pour votre attention – Questions ?